

Máquinas Universais e Variantes da Máquina de Turing

APD, λ -Cálculo e MT Não Determinística

Teoria da Computabilidade



CESUPA

Turma CC5MA

Professor

Daniel Leal Souza

Integrantes da equipe

1. João Pedro Almeida Follmann
2. Samuel Paula Nunes Salheb
3. Yuri Antonio Santos Fernandes
4. Arthur José Aviz Lima
5. Gabriel Cruz Filgueira

Visão geral do projeto

Modelo	Problema resolvido	Implementação
APD	Reconhecimento de expressões aritméticas válidas	JFLAP (.jff)
λ -Cálculo	Calculadora funcional com Numerais de Church	Jupyter Notebook
MTND	Subset Sum em unário	Python

Mensagem principal

Cada modelo resolve um problema diferente e demonstra uma forma distinta de computação.

Autômato com Pilha — Expressões Aritméticas

- Controle finito + pilha
- Pilha controla abertura e fechamento de parênteses
- Reconhece variáveis, operadores e parênteses

Linguagem reconhecida

variáveis: a, b, c
operadores: +, -, *, /
parênteses: ()

Exemplos de entrada

aceita

a+b

aceita

(a+b)

rejeita

a+

Ideia: depois de operador, deve vir operando; parênteses precisam fechar corretamente.

Formalização resumida do APD

Definição formal

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$

$Q = \{q_0, q_1, \dots, q_9\}$

$\Sigma = \{a, b, c, +, -, *, /, (,)\}$

$\Gamma = \{Z, P\}$

q_0 = estado inicial

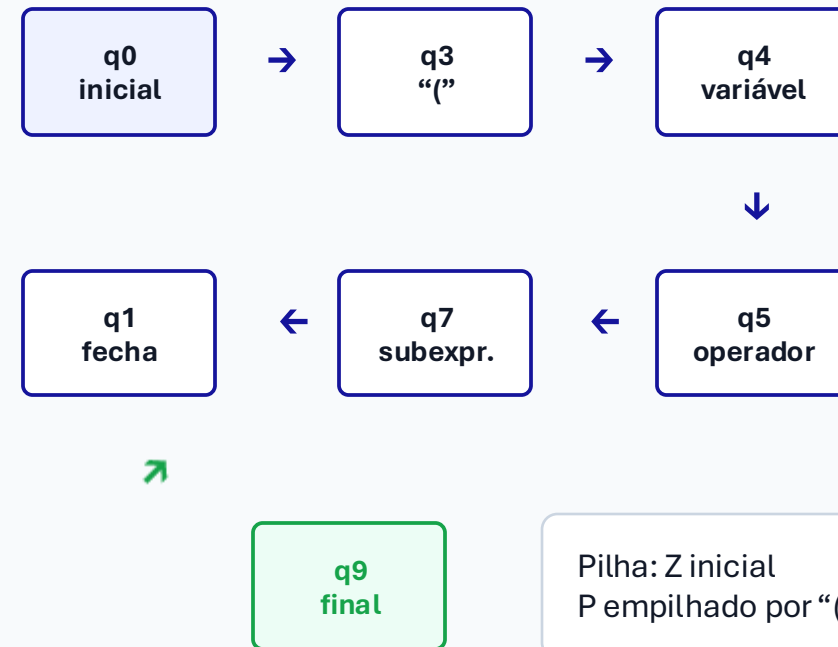
Z = símbolo inicial da pilha

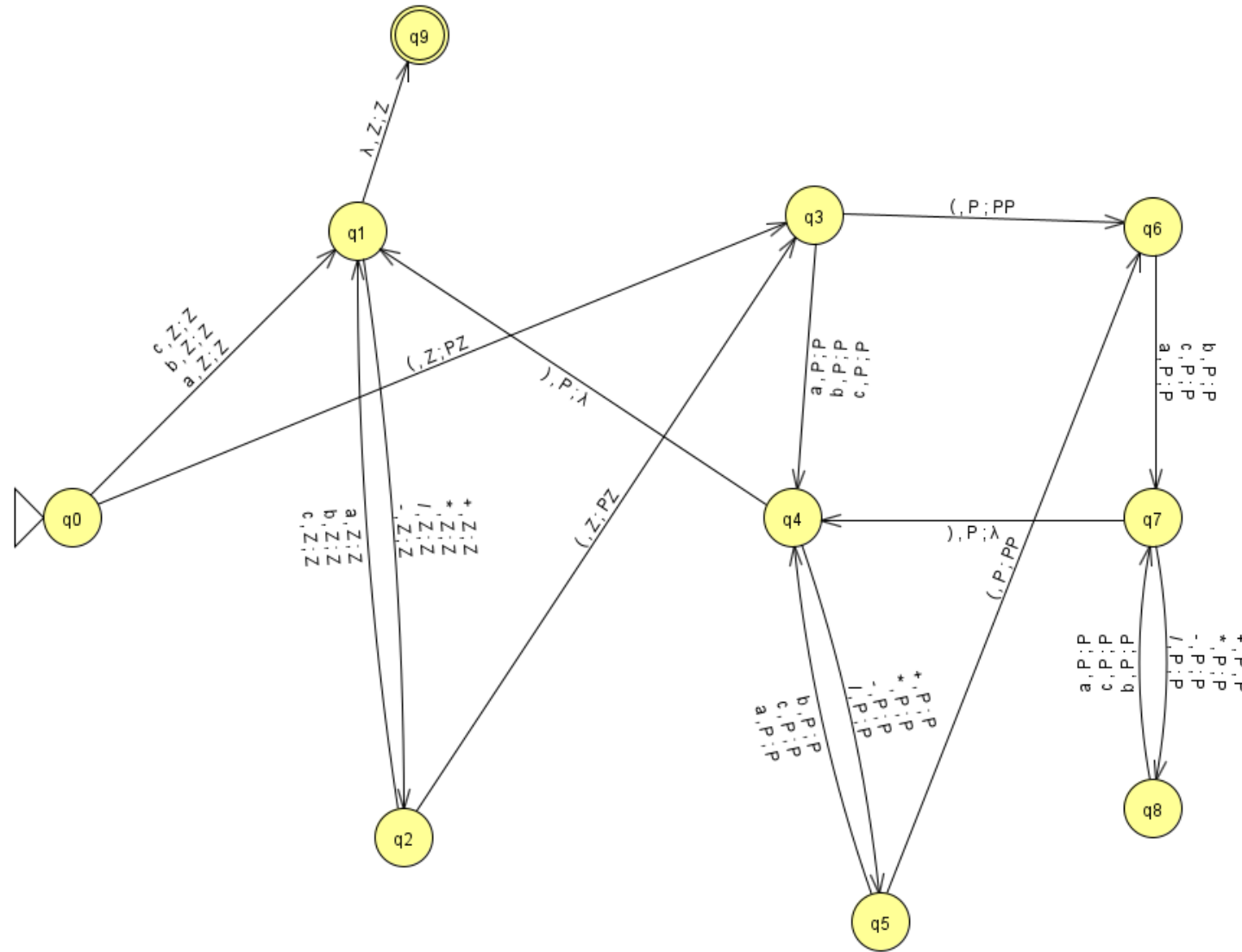
$F = \{q_9\}$

Critério

Aceitação por estado final, com entrada consumida.

Representação simplificada da máquina (.jff)





Máquina no JFLAP

Rastreamento: entrada (a+b)

Passo	Estado	Entrada restante	Pilha	Ação
1	q0	(a+b)	Z	lê "(" e empilha P
2	q3	a+b)	PZ	lê variável
3	q4	+b)	PZ	lê operador
4	q5	b)	PZ	lê variável
5	q4	)	PZ	desempilha P
6	q1	ϵ	Z	vai para q9
7	q9	ϵ	Z	aceita

Resultado: estado final q9 com entrada consumida.

λ-Cálculo — Numerais de Church

- Modelo baseado em funções puras
- Computação por redução β
- Não usa números primitivos
- Números são funções

Numerais de Church

$$0 \equiv \lambda f. \lambda x. x$$

$$1 \equiv \lambda f. \lambda x. f x$$

$$2 \equiv \lambda f. \lambda x. f (f x)$$

Operações implementadas

SUCC



ADD



MULT

Demonstração: expressão reduzida até forma normal.

Avaliador em Python

Estrutura do avaliador

- representação por AST
- classes Var, Abs e App
- estratégia: ordem normal
- substituição captura-evitante
- conversão α quando necessária



```

def passo(t):
    if isinstance(t, App):
        if isinstance(t.func, Abs):
            return subst(t.func.body,
                          t.func.param,
                          t.arg)
    ...
  
```

Quando uma abstração é aplicada, ocorre a redução β .

Rastreamento: ADD UM DOIS

Entrada

ADD UM DOIS

Resultado

$\lambda f. \lambda x. f (f (f x))$

Interpretação

$1 + 2 = 3$

Expressão	Passos	Forma normal	Resultado
SUCC ZERO	3	$\lambda f. \lambda x. f x$	1
ADD UM DOIS	7	$\lambda f. \lambda x. f (f (f x))$	3
MULT DOIS TRES	12	$\lambda f. \lambda x. f (f (f (f (f (f x)))))$	6

Máquina de Turing Não Determinística — Subset Sum

- Múltiplas transições possíveis
- Computação forma uma árvore de escolhas
- Aceita se algum ramo chega ao estado de aceitação

Problema

Dado um conjunto em unário e um alvo, verificar se existe subconjunto cuja soma seja igual ao alvo.

Entrada exemplo

aa#aaa#a=aaaa

{2, 3, 1}, alvo 4

Resultado

Aceita, pois $3 + 1 = 4$.



Formalização resumida da MTND

Estado	Função
q0	valida entrada
q1	escolhe próximo número
q2	inclui número
q3	ignora número
q4	compara soma com alvo
qacc	aceita
qrej	rejeita

Configuração

(estado, números restantes, soma atual, alvo, caminho)

Transição não determinística

Para cada número:

ramo 1: inclui o número

ramo 2: ignora o número

Rastreamento: aa#aaa#a=aaaa

```
soma 0
├─ inclui 2 → soma 2
├─ inclui 3 → soma 5
├─ ignora 3 → soma 2
├─ ignora 2 → soma 0
├─ inclui 3 → soma 3
├─ inclui 1 → soma 4 → aceita
├─ ignora 1 → soma 3 → rejeita
├─ ignora 3 → soma 0
```

Ramo aceitante

ignora 2
inclui 3
inclui 1

Critério

um único ramo
aceitante basta.

Testes das três implementações

Modelo	Entrada	Resultado esperado	Resultado obtido
APD	a+b	aceita	aceita
APD	a+	rejeita	rejeita
λ-Cálculo	ADD UM DOIS	3	3
λ-Cálculo	MULT DOIS TRES	6	6
MTND	aa#aaa#a=aaaa	aceita	aceita
MTND	aa#aaaa=aaa	rejeita	rejeita

Objetivo dos testes: confirmar aceitação, rejeição e rastreamento esperado.

Comparação técnica

Modelo	Memória / mecanismo	O que demonstra	Limitação
APD	Pilha	Reconhecimento sintático	Não avalia expressão
λ -Cálculo	Substituição funcional	Computação por funções puras	Pode não terminar
MTND	Ramos de computação	Escolha não determinística	Crescimento exponencial da árvore

Leitura final

APD mostra sintaxe com pilha; λ -Cálculo mostra funções; MTND mostra busca por ramos.

OBIGADO